Autotokamak: An Agentic Platform for Learning Fast Surrogates of the Grad–Shafranov Equilibrium

A System Description and Mathematical Reference

Abstract

Autotokamak is a research platform that learns fast machine-learning surrogates for the Grad–Shafranov (GS) plasma-equilibrium problem and wraps the entire learning process in an LLM-driven agentic loop that plans, runs, diagnoses, and improves the pipeline autonomously. The ground-truth solver is TokaMaker (from the OpenFUSION Toolkit), a finite-element solver for the axisymmetric GS equation. Autotokamak treats that solver as an expensive black-box forward operator $\mathcal{F} : \mathbf{x} \mapsto \psi$ mapping five scalar shaping/current parameters to a 2-D poloidal-flux field, and asks: *can we learn a cheap approximation $\hat{\mathcal{F}} \approx \mathcal{F}$, choosing where to spend expensive solves on purpose, and can an LLM agent orchestrate that whole workflow?* This document is written to be read linearly by someone new to the system: each section pairs an intuitive explanation and a diagram with the precise mathematics. We cover the physics forward problem, the principal-component + Gaussian-process regression surrogate, the AutoML hyperparameter optimization, the residual-driven active-learning acquisition, and the outer meta-optimization loop.

Contents

1 Introduction

1.1 The problem in one paragraph

A tokamak confines plasma in a doughnut-shaped magnetic cage. Designing and controlling one requires knowing the *equilibrium* — the shape of the magnetic surfaces that balance the plasma pressure against the magnetic forces. That equilibrium is the solution of a partial differential equation (the Grad–Shafranov equation), and computing it accurately means running a finite-element solver that can take seconds to minutes per configuration. Anyone who wants to explore thousands of machine designs, or control a plasma in real time, cannot afford that cost. Autotokamak's central idea is therefore simple to state: *learn a fast approximation of the expensive solver*, and be smart about which expensive solves you spend your budget on while building that approximation.

1.2 Two research threads, one platform

Autotokamak pursues two complementary goals under one codebase:

1. **ML surrogate models** for the GS equation — fast approximations of TokaMaker's FEM solve, so that a flux field costing seconds–minutes of FEM can be predicted in milliseconds.
2. **LLM-driven agentic workflows** (built on the URSA plan/execute framework) that plan and run equilibrium computations and the surrogate-learning process end-to-end, choosing what to compute next based on *observed* model performance rather than a fixed script.

The first thread is a machine-learning contribution; the second is an automation contribution. They meet in the Phase-3 meta-loop (Section 6), where an LLM agent decides, at each step, whether the

surrogate needs more data, better data, or a better model.

1.3 The shape of the system

Before any mathematics, Figure 1 gives the whole system at a glance. The platform is deliberately split into two layers. **Layer 1** is the stable numerical substrate: physics, geometry, the solver, diagnostics, and file I/O. It is deterministic, reproducible, and never touched by the language model. **Layer 2** is the orchestration layer: the URSA plan/execute agents, the meta-loop, and the DSPy/GEPA-optimized decision modules. The language model lives entirely in Layer 2 and only ever makes *high-level choices*; every load-bearing number (a kernel, a variance, a cross-validation fold) is computed by Layer 1 library code. The two layers communicate through a handful of on-disk artifacts — a dataset, a trained model, a trace.

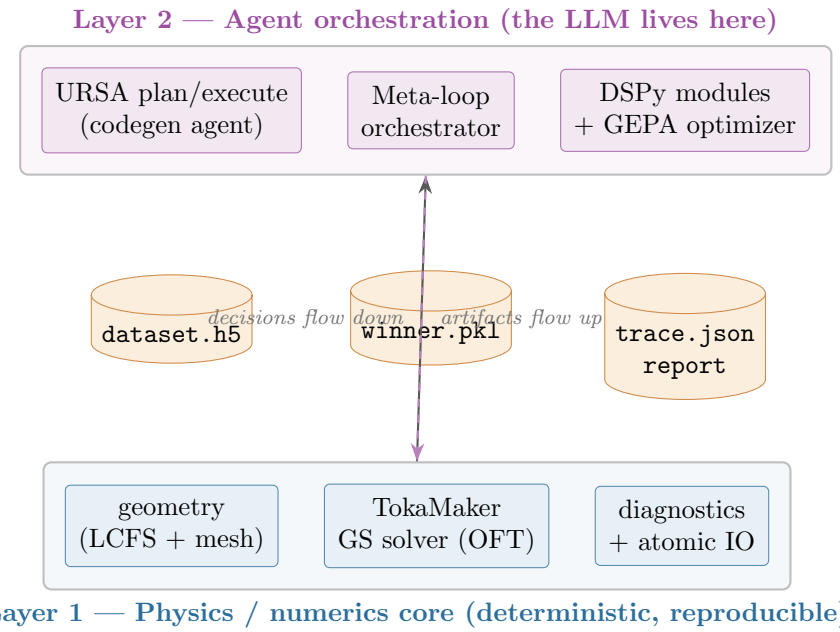

Figure 1: The two-layer architecture. The language model makes only high-level decisions in Layer 2; all numerics are deterministic Layer 1 library code. The layers exchange only on-disk artifacts, which keeps the science reproducible and the solver's process-singleton constraint isolated.

1.4 Three phases

The learning pipeline runs in three phases, which the outer agent may revisit in any order (Figure 2). Phases 1–2 are the classic "generate data, then fit a model" loop. Phase 3 is the contribution that closes the loop: it *measures where the current surrogate is weak* and directs new expensive solves at exactly those regions, and it hands the high-level "what should we do next?" decision to an LLM.

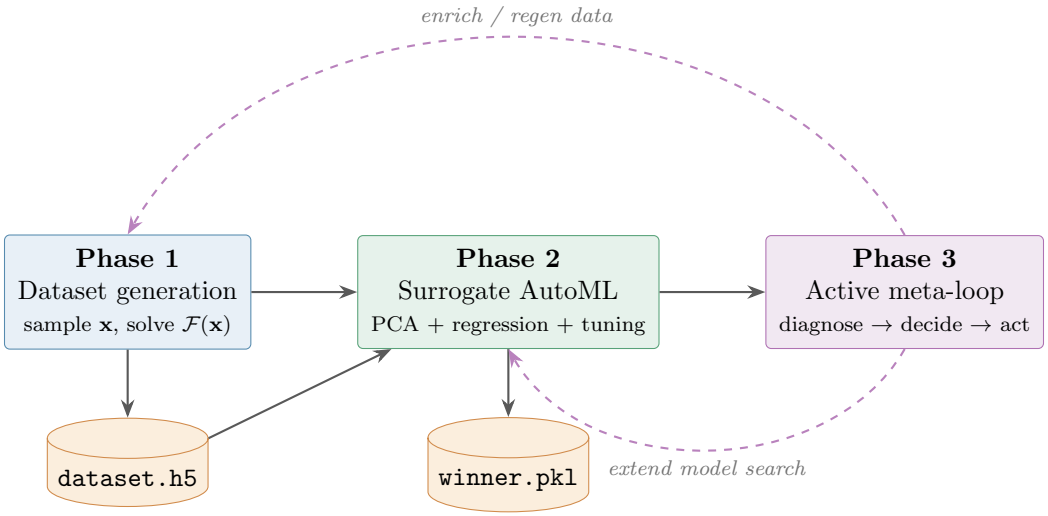

Figure 2: The three phases and the feedback the meta-loop adds. Solid arrows are the forward pipeline; dashed arrows are the Phase-3 decisions that send the loop back to acquire better data (Phase 1) or search a better model (Phase 2).

1.5 Notation

Symbol	Meaning
$\mathbf{x} = (R_0, a, \kappa, \delta, I_p) \in \mathbb{R}^5$	tokamak shaping/current parameters (input)
$\psi(R, Z)$	poloidal flux field (output), discretized to $\Psi \in \mathbb{R}^{n_Z \times n_R}$
$\mathcal{F}$	the true GS forward operator (TokaMaker)
$\hat{\mathcal{F}}$	the learned surrogate
$P = n_Z\, n_R$	number of output pixels ($96 \times 64 = 6144$ in the shipped grid)
k	number of retained PCA components
$\mathcal{D} = \{(\mathbf{x}_i, \Psi_i)\}_{i=1}^{N}$	dataset of solved equilibria

2 The physics forward problem

2.1 What we are computing, and why it is expensive

The object of interest is the **poloidal flux** $\psi(R, Z)$, a scalar field on a 2-D slice through the torus. Its contour lines are the magnetic surfaces on which the plasma sits, so ψ *is* the equilibrium: from it one derives the plasma boundary, the safety factor, the location of the magnetic axis, and every stability metric an engineer cares about. Computing ψ means solving a nonlinear elliptic PDE on a finite-element mesh, iterating until the plasma current and the magnetic field it produces are self-consistent. That self-consistency iteration is what makes each solve expensive — and what makes a learned surrogate worthwhile.

2.2 The Grad–Shafranov equation

TokaMaker solves the axisymmetric ideal-MHD force-balance equation — the **Grad–Shafranov equation** — for $\psi(R, Z)$ in the 2-D poloidal (R, Z) plane:

$$\Delta^* \psi \;=\; -\,\mu_0\, R^2\, p'(\psi) \;-\; F(\psi)\, F'(\psi). \tag{1}$$

Reading the equation term by term: the left-hand side is a differential operator acting on the unknown field ψ; the right-hand side is the *source* that the plasma's own pressure and current supply. The two sources are what make it nonlinear — they depend on ψ, the very thing being solved for. Concretely, $\psi(R, Z)$ is the poloidal flux function, whose level sets $\{\psi = \text{const}\}$ are the nested magnetic flux surfaces; Δ^* is the **Grad–Shafranov operator**, a non-Laplacian elliptic operator that in cylindrical coordinates reads

$$\Delta^*\psi \;=\; R\,\frac{\partial}{\partial R}\left(\frac{1}{R}\frac{\partial \psi}{\partial R}\right) + \frac{\partial^2 \psi}{\partial Z^2}, \tag{2}$$

the extra $1/R$ factor (relative to the ordinary Laplacian) arising from the toroidal geometry. $p(\psi)$ is the plasma pressure profile with $p' = \mathrm{d}p/\mathrm{d}\psi$; $F(\psi) = R\,B_\phi$ is the poloidal-current function (B_ϕ = toroidal field) with $F' = \mathrm{d}F/\mathrm{d}\psi$; and μ_0 is the vacuum permeability. Because the right-hand side is nonlinear in ψ, TokaMaker solves it by Picard/Newton iteration on a triangular finite-element mesh.

2.3 Boundary: the Last Closed Flux Surface

An elliptic PDE needs a boundary condition. In **fixed-boundary** mode (this platform's default) we prescribe the outermost closed contour — the **Last Closed Flux Surface (LCFS)** — and solve for ψ inside it. The LCFS is an analytic D-shaped curve; a standard tokamak parameterization is

$$\begin{aligned} R(\theta) &= R_0 + a\cos\big(\theta + \delta\sin\theta\big), \\ Z(\theta) &= Z_0 + \kappa\, a \sin\theta, \qquad \theta \in [0, 2\pi), \end{aligned} \tag{3}$$

with R_0 the major radius of the plasma center (m), a the minor radius / half-width (m), κ the elongation (1 = circular, ≈ 1.6 ITER-like), δ the triangularity (0 = symmetric, > 0 = D-shaped), and Z_0 the vertical center (fixed to 0 here). These are four of our five input knobs; the fifth, the plasma current I_p, sets the overall magnitude of ψ.

An **isoflux** constraint additionally pins ψ to a constant along the sampled boundary points, $\psi(R(\theta_j), Z(\theta_j)) = \psi_b \;\forall j$. It is more accurate but numerically fragile: on extreme shapes it can fail at construction, in which case the solver drops the constraint and re-solves unconstrained. Whether isoflux held is recorded per sample — an important bookkeeping detail, because *if it fell back, the geometry inputs no longer describe the saved* ψ and that sample must be flagged and excluded from clean training.

2.4 The forward operator and its feasible set

For the learning problem we hide all of that machinery behind a single deterministic forward operator

$$\mathcal{F}:\; \mathbf{x} = (R_0, a, \kappa, \delta, I_p) \;\longmapsto\; \psi(R, Z). \tag{4}$$

The output is interpolated from the FEM mesh onto a fixed rectangular grid $\Psi \in \mathbb{R}^{n_Z \times n_R}$ so downstream ML can treat it as an image tensor (Figure 3 makes this concrete). Pixels *outside the LCFS* carry no plasma and are set to NaN; all metrics mask them out.

A subtlety that shapes the whole active-learning design: not every $\mathbf{x}$ produces a valid solve. Extreme shaping (high κ, high δ, small a) drives the solver into isoflux fallback or outright failure. We therefore treat the solver as a *constrained* black box with an unknown **feasible set**

$$\Omega = \{\mathbf{x} : \mathcal{F}(\mathbf{x}) \text{ succeeds}\} \subseteq \mathcal{X}, \tag{5}$$

and later learn a probabilistic model of Ω (Section 5.4) so we do not waste budget solving points that will fail. The shipped seed box is $R_0 \in [0.35, 0.55]$ m, $a \in [0.10, 0.20]$ m, $\kappa \in [1.0, 1.6]$, $\delta \in [0, 0.4]$, $I_p \in [80, 200]$ kA.

3 Phase 1 — Dataset generation

3.1 Choosing where to sample

A supervised model needs examples, and each example here is one expensive solve. With a fixed budget of N solves, *where* in the 5-D box we place them matters. Blind sampling uses a space-filling design — either **Latin Hypercube Sampling (LHS)** or a **Sobol** low-discrepancy sequence — rather than independent uniform draws. The reason is coverage: for a fixed N, these designs spread points across the box far more evenly (they have lower star-discrepancy), so no region is left accidentally empty and none is wastefully clustered. Every draw is seeded, so a dataset can be regenerated bit-for-bit. Phase 3 will later replace this "blind" placement with a *targeted* one, but blind space-filling is the right way to seed.

3.2 The dataset tensor

The HDF5 artifact stores, for N samples: `inputs`$\in \mathbb{R}^{N\times 5}$ (columns $(R_0, a, \kappa, \delta, I_p)$); `psi`$\in \mathbb{R}^{N\times n_Z\times n_R}$ (fields, NaN outside the LCFS); `R`,`Z` (grid axes); `success`$\in \{0,1\}^N$ (whether the isoflux solve succeeded — this is our labelled data for the feasibility model); and provenance for reproducibility. One design decision is worth stating because it drives a downstream parameter: the stored ψ is **physical poloidal flux in Webers**, not per-sample normalized $\psi_N \in [0, 1]$. Normalizing each field to $[0, 1]$ would erase the I_p dependence entirely (the current only sets the field's scale), which is information the surrogate must predict. Keeping physical units means the data is *less* low-rank (Section 4.2), so we retain more principal components than a normalized dataset would need.

4 Phase 2 — The surrogate model

4.1 The idea: don't predict 6144 pixels directly

We want a cheap map $\hat{\mathcal{F}}_\theta$ that reproduces the solver's field, formally minimizing expected field error over the input distribution $p(\mathbf{x})$:

$$\theta^\star = \arg\min_\theta \ \mathbb{E}_{\mathbf{x}\sim p}\Big[\ \big\|\, \mathcal{F}(\mathbf{x}) - \hat{\mathcal{F}}_\theta(\mathbf{x}) \,\big\|^2_{\text{masked}}\ \Big], \tag{6}$$

where $\|\cdot\|_{\text{masked}}$ is the RMS over in-LCFS pixels and, with only the finite sample $\mathcal{D}$, we minimize a cross-validated estimate of this loss. The output is a $96 \times 64 = 6144$-pixel image, and regressing 6144 correlated targets directly from five inputs is both wasteful and noisy. The key observation is that equilibrium flux fields are extremely redundant: across a whole dataset they nearly live on a low-dimensional linear subspace. So the surrogate **factors into two steps** (Figure 3): compress the field to a few numbers with PCA, then regress those few numbers from the five inputs.

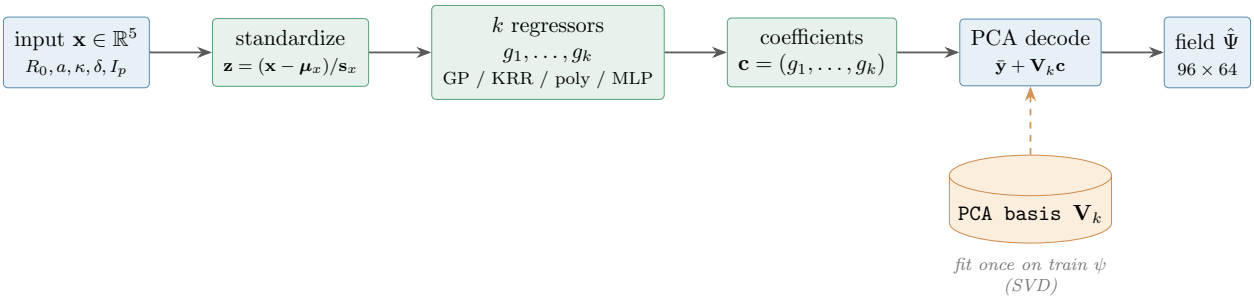

Figure 3: The surrogate factorization. Rather than predict 6144 pixels directly, PCA compresses the flux field to k coefficients (learned once from the training fields by SVD); k small regressors map the five standardized inputs to those coefficients; and the orthonormal PCA basis $\mathbf{V}_k$ decodes them back to a full field.

4.2 Output reduction: Principal Component Analysis

Flatten each field to $\mathbf{y}_i \in \mathbb{R}^P$ (NaN pixels imputed by the per-pixel training mean $\bar{\mathbf{y}}$), stack the centered fields into $\mathbf{Y}_c = [\mathbf{y}_1 - \bar{\mathbf{y}}, \ldots]^\top \in \mathbb{R}^{N \times P}$, and take its singular value decomposition

$$\mathbf{Y}_c = \mathbf{U}\,\boldsymbol{\Sigma}\,\mathbf{V}^\top. \tag{7}$$

The first k right-singular vectors $\mathbf{V}_k = [\mathbf{v}_1, \ldots, \mathbf{v}_k] \in \mathbb{R}^{P \times k}$ form an orthonormal basis ($\mathbf{V}_k^\top \mathbf{V}_k = \mathbf{I}_k$) of the retained flux subspace — think of each $\mathbf{v}_j$ as an "eigen-equilibrium", a characteristic flux pattern, and of any field as a weighted sum of a handful of them. Encoding and reconstruction are simply projections onto and out of that basis:

$$\underbrace{\mathbf{c}_i = \mathbf{V}_k^\top(\mathbf{y}_i - \bar{\mathbf{y}})}_{\text{encode } (P \to k)} \qquad \underbrace{\hat{\mathbf{y}}_i = \bar{\mathbf{y}} + \mathbf{V}_k\,\mathbf{c}_i}_{\text{decode } (k \to P)}. \tag{8}$$

How many components we keep is governed by how much field variance each explains:

$$\mathrm{EVR}_j = \frac{\sigma_j^2}{\sum_l \sigma_l^2}, \qquad \text{cumulative EV}(k) = \sum_{j=1}^{k} \mathrm{EVR}_j, \tag{9}$$

and k is chosen (or searched) to reach a target cumulative explained variance. Because we keep physical ψ (Section 4.2), the data is *less* low-rank than normalized data: ≈ 8 components reach $\sim$ 85% EV, versus $\sim$ 99% for normalized data — so k (and the acquisition PCA size) is set accordingly.

4.3 Latent-space regression: the model zoo

With the field reduced to k coefficients, the learning task is k ordinary scalar regressions, $g_j : \mathbb{R}^5 \to \mathbb{R}$, one per coefficient. Assembling them and decoding gives the full surrogate:

$$\boxed{\hat{\mathcal{F}}(\mathbf{x}) \;=\; \bar{\mathbf{y}} \;+\; \sum_{j=1}^{k} g_j(\mathbf{x})\,\mathbf{v}_j} \tag{10}$$

reshaped to the $n_Z \times n_R$ grid. Inputs are *always* standardized first, $\mathbf{z} = (\mathbf{x} - \boldsymbol{\mu}_x)/\mathbf{s}_x$, for a concrete reason: the raw features span wildly different scales ($I_p \sim 10^5$ versus $R_0 \sim 0.4$), so without standardization every kernel distance is dominated by I_p alone and the model collapses to predicting the mean. Four regression families ("the zoo") compete for each dataset; the agent chooses among them.

(a) Gaussian Process (GP) regression — the headline model. A GP is the natural choice here because it is data-efficient at small N *and* returns a calibrated uncertainty that the active-learning stage will reuse. It uses a squared-exponential (RBF) kernel plus a white-noise term,

$$k(\mathbf{z}, \mathbf{z}') \;=\; \sigma_f^2 \exp\!\left(-\frac{\|\mathbf{z}-\mathbf{z}'\|^2}{2\ell^2}\right) \;+\; \sigma_n^2\,\delta_{\mathbf{z}\mathbf{z}'}, \tag{11}$$

which encodes the belief that nearby inputs give similar outputs, with ℓ setting "how near". Placing a Gaussian prior over functions and conditioning on the training data $(\mathbf{Z}, \mathbf{c}^{(j)})$ yields a Gaussian **posterior predictive** at a new $\mathbf{z}_\star$ with closed-form mean and variance:

$$g_j(\mathbf{z}_\star) \;=\; \mu_j(\mathbf{z}_\star) = \mathbf{k}_\star^\top\,(\mathbf{K}+\alpha\mathbf{I})^{-1}\,\mathbf{c}^{(j)}, \tag{12}$$

$$\sigma_j^2(\mathbf{z}_\star) = k(\mathbf{z}_\star,\mathbf{z}_\star) - \mathbf{k}_\star^\top\,(\mathbf{K}+\alpha\mathbf{I})^{-1}\,\mathbf{k}_\star, \tag{13}$$

where $\mathbf{K}_{ab} = k(\mathbf{z}_a, \mathbf{z}_b)$ is the Gram matrix over training inputs, $\mathbf{k}_\star$ collects the covariances between $\mathbf{z}_\star$ and the training points, and α is a jitter/regularizer. The prediction is a *data-weighted average of observed coefficients*; the variance grows away from the data — which is exactly the "I am unsure here" signal we later exploit. The kernel hyperparameters $(\sigma_f, \ell, \sigma_n)$ are fit by maximizing the **log marginal likelihood**

$$\log p(\mathbf{c}^{(j)} \mid \mathbf{Z}) = -\tfrac{1}{2}\,\mathbf{c}^{(j)\top}\mathbf{K}_y^{-1}\mathbf{c}^{(j)} - \tfrac{1}{2}\log|\mathbf{K}_y| - \tfrac{N}{2}\log 2\pi, \quad \mathbf{K}_y = \mathbf{K} + \sigma_n^2\mathbf{I}, \tag{14}$$

which trades data fit (first term) against model complexity (second term) automatically.

(b) Kernel Ridge Regression (KRR). Same kernel intuition, no uncertainty, cheaper to fit — useful when the GP's likelihood optimization is unstable at tiny N:

$$g_j(\mathbf{z}_\star) = \mathbf{k}_\star^\top\boldsymbol{\eta}, \qquad \boldsymbol{\eta} = (\mathbf{K}+\lambda\mathbf{I})^{-1}\mathbf{c}^{(j)}, \tag{15}$$

with kernel $\in \{\text{RBF}, \text{Laplacian}\}$, regularizer λ (`alpha`), bandwidth `gamma`.

(c) Polynomial ridge. An explicit polynomial feature map $\boldsymbol{\phi}(\mathbf{z})$ of degree $d \in \{1,2,3\}$ into ridge regression — interpretable and fast, with a low-frequency bias that suits smooth fields:

$$\mathbf{w}_j = \arg\min_{\mathbf{w}}\; \|\boldsymbol{\Phi}\mathbf{w} - \mathbf{c}^{(j)}\|^2 + \lambda\|\mathbf{w}\|^2 = (\boldsymbol{\Phi}^\top\boldsymbol{\Phi} + \lambda\mathbf{I})^{-1}\boldsymbol{\Phi}^\top\mathbf{c}^{(j)}, \qquad g_j(\mathbf{z}_\star) = \boldsymbol{\phi}(\mathbf{z}_\star)^\top\mathbf{w}_j. \tag{16}$$

(d) Small MLP. A capped multilayer perceptron (≤ 2 hidden layers, width ≤ 256) — a universal approximator kept small to avoid overfitting the modest dataset, trained with L2 penalty α and a chosen learning rate. Deep operator/PINN/FNO families are explicitly out of the proof-of-concept scope, so the zoo is intentionally classical and CPU-cheap.

4.4 AutoML: automatically tuning each model

Each model has knobs (a length scale, a regularizer, a network width) whose best values are not known in advance, so they are *searched*. The objective is the honest one — **K-fold cross-validated field RMSE in physical ψ units**, with the PCA refit inside each fold to avoid leaking test information into the basis:

$$\mathcal{L}(\mathbf{h}) = \frac{1}{K}\sum_{f=1}^{K}\text{RMSE}\Big(\Psi_{\text{va}_f},\; \hat{\mathcal{F}}_{\mathbf{h}}^{(\text{tr}_f)}(\mathbf{X}_{\text{va}_f})\Big), \quad \text{RMSE}(\Psi,\hat{\Psi}) = \sqrt{\frac{1}{|\mathcal{M}|}\sum_{(i,p)\in\mathcal{M}}(\Psi_{ip}-\hat{\Psi}_{ip})^2}, \tag{17}$$

where $\mathcal{M}$ is the set of finite (in-LCFS) cells. The search itself solves

$$\mathbf{h}^\star = \arg\min_{\mathbf{h}\in\mathcal{H}} \ \mathcal{L}(\mathbf{h}) \tag{18}$$

using **Optuna's Tree-structured Parzen Estimator (TPE)** — a Bayesian-optimization sampler that, instead of a grid, learns from past trials which regions of $\mathcal{H}$ produce good scores. It models the density of hyperparameters among the good trials, $\ell(\mathbf{h})$, and among the poor ones, $g(\mathbf{h})$, and proposes points maximizing the ratio $\ell(\mathbf{h})/g(\mathbf{h})$ (an expected-improvement surrogate over the *configuration* space, distinct from the GP surrogate over the *physics*). A trial that crashes returns a large sentinel objective (10^{10}) so the search simply moves on. When the search ends, the best $(\text{model}, \mathbf{h}^\star, k)$ is **refit on all non-test samples** and saved as `winner.pkl` with its PCA handle.

4.5 Did it actually learn anything? Scoring against a baseline

A low RMSE means little on its own — a field with small magnitude has small errors trivially. We therefore always compare against the **trivial baseline**: predict every field as the per-pixel mean of the training fields, $\hat{\Psi}^{\text{base}} = \bar{\Psi}_{\text{train}}$. The two headline numbers are relative to that baseline:

$$\text{RMSE ratio} = \frac{\text{RMSE}(\hat{\mathcal{F}})}{\text{RMSE}(\hat{\Psi}^{\text{base}})}, \qquad \text{error reduction} = 100\big(1 - \text{RMSE ratio}\big)\ \%, \tag{19}$$

so "90% accuracy" means the surrogate cuts the baseline field error by 90%, and any acceptable winner must have RMSE ratio < 1 (i.e. beat doing nothing). Additional diagnostics — MAE, relative L_2, R^2, correlation, tolerance-band accuracy — are reported for interpretation.

5 Phase 3 — Active learning: sampling where the model is weak

5.1 Why blind sampling wastes budget

Blind LHS spends the same effort everywhere, but a surrogate is rarely uniformly bad — it is usually accurate in the well-sampled middle of the box and poor in some corner (say, high elongation). If our next batch of expensive solves is scattered blindly, most of it lands where the model is *already* good and teaches it little. Active learning fixes this: it uses the current model's own errors to decide where the next solves should go, so the budget is spent closing the weak regions. Figure 4 shows the loop; the default strategy is **residual-driven Upper Confidence Bound (residual-UCB)**, with graceful fallbacks when a trustworthy winner does not yet exist.

5.2 The acquisition signal: out-of-fold residuals

To target weakness we must first *measure* it honestly. The signal is the winner's **out-of-fold (OOF) residual** on the training pool: for each cross-validation fold we refit the winner's architecture on the fold's training rows and record each held-out sample's mean absolute field error,

$$r_i \ = \ \frac{1}{|\mathcal{M}_i|} \sum_{p\in\mathcal{M}_i} \big|\, \Psi_{ip} - \hat{\mathcal{F}}^{(-\text{fold}(i))}(\mathbf{x}_i)_p \,\big|. \tag{20}$$

Using OOF (not training-set) residuals is essential: interpolating models such as GP and KRR fit their own training points to ≈ 0 error, so training residuals would report "perfect everywhere" and carry no signal about *generalization*. The OOF residual, by contrast, measures error at points the fitted model has never seen — exactly the quantity we want to reduce.

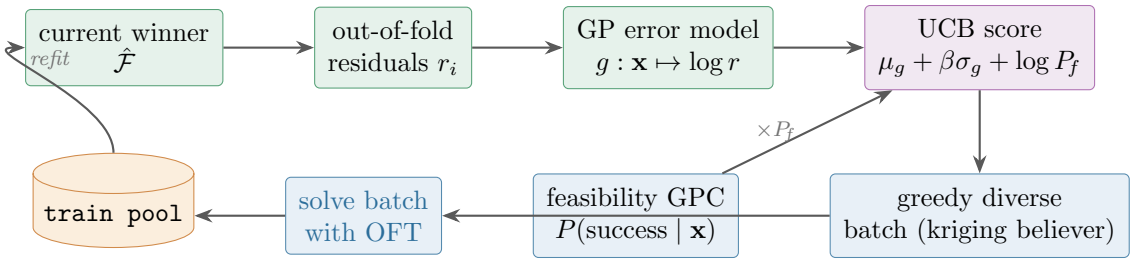

Figure 4: The residual-driven active-learning loop. The winner's out-of-fold residuals train a GP *error model*; a UCB acquisition (discounted by a feasibility classifier) ranks a candidate pool; a greedy "kriging-believer" rule picks a spread-out batch; those points are solved, merged into the pool, and the winner is refit — closing the loop.

5.3 The error model and the UCB score

We then fit a *second* GP — the **error model** — to predict how large the residual will be at any candidate geometry, working with log residuals over standardized inputs,

$$g : \mathbf{x} \longmapsto \log r, \qquad g \sim \mathcal{GP}\big(\mu_g,\, k_{\text{ARD}}\big), \tag{21}$$

using an **ARD (Automatic Relevance Determination) RBF kernel** with a separate length scale per input dimension,

$$k_{\text{ARD}}(\mathbf{z}, \mathbf{z}') = \sigma_f^2 \exp\left(-\tfrac{1}{2} \sum_{d=1}^{5} \frac{(z_d - z'_d)^2}{\ell_d^2} \right) + \sigma_n^2 \delta. \tag{22}$$

Per-dimension length scales let the error model discover that weakness aligns with a few input directions (e.g. elongation) and ignore the irrelevant ones. The log transform reflects that error is multiplicatively structured and keeps a few catastrophic samples from dominating the fit. Like any GP, this one gives a posterior mean $\mu_g(\mathbf{x})$ (predicted error) and standard deviation $\sigma_g(\mathbf{x})$ (confidence in that prediction).

Now comes the central decision rule. We do not simply solve wherever predicted error is highest — that would ignore regions the error model has never seen. Instead we rank candidates by an **Upper Confidence Bound**, written in log space so the feasibility term adds in cleanly:

$$a(\mathbf{x}) \;=\; \underbrace{\mu_g(\mathbf{x})}_{\text{exploit measured weakness}} \;+\; \beta\, \underbrace{\sigma_g(\mathbf{x})}_{\text{explore the unknown}} \;+\; \underbrace{\log P_{\text{feas}}(\mathbf{x})}_{\text{avoid infeasible solves}} \tag{23}$$

The first term *exploits*: go where we have measured the model to be bad. The second term *explores*: go where the error model itself is uncertain — which naturally includes regions with no data yet, such as a target envelope wider than the seed box. The weight $\beta \in [0, 3]$, exposed to the meta-agent, tunes the balance (Figure 5): $\beta = 0$ is pure exploitation and may pile onto one weak pocket; larger β pushes toward coverage. The third term discounts candidates likely to fail. Candidates are drawn from a large Sobol pool over the envelope bounds.

5.4 Feasibility weighting: modeling the feasible set Ω

A failed solve costs the same wall-clock as a successful one, so it is worth learning which geometries fail. A GP classifier (GPC) with an ARD-RBF kernel is trained on *every attempted input*, successes

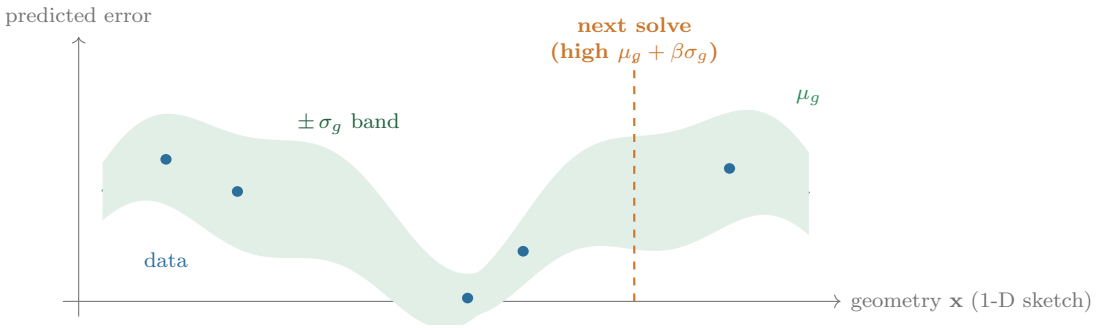

Figure 5: The exploit/explore intuition behind the UCB (schematic, 1-D). The green curve is the error model's predicted error μ_g; the band is its uncertainty $\pm\sigma_g$, which widens away from the sampled points (blue). The acquisition prefers points that are both predicted-bad *and* uncertain, so the next solve lands where the model is most likely weak yet under-explored — not merely at the tallest known peak.

and failures alike (this is why Phase 1 records the `success` flag), giving $P(\text{success} \mid \mathbf{x})$. The acquisition is discounted by the *squared* probability, floored:

$$P_{\text{feas}}(\mathbf{x}) \;=\; \max\Big(P(\text{success} \mid \mathbf{x})^2,\; \varepsilon \Big), \qquad \varepsilon = 0.02. \tag{24}$$

Squaring prevents a known-bad region from out-bidding a merely well-covered feasible one on prior variance alone; the (deliberately harsh) floor ε keeps an unlucky region down-weighted but never permanently written off. If the GPC fails to fit, a distance-weighted k-NN estimate is used instead. ARD matters here for the same reason as before: failures align with a few shaping directions, and an isotropic distance would let the irrelevant dimensions blur that boundary.

5.5 Batch selection: picking a spread-out batch, not one point n times

We usually want a *batch* of n new solves at once, and naïvely taking the top-n acquisition scores would return n near-duplicates clustered on a single peak. The fix rests on a special property of GPs: **the posterior variance depends only on *where* you have observed, not on *what* you observed**. So under the "kriging-believer" heuristic we pretend each pick was observed at the model's own mean — which leaves μ_g unchanged but shrinks σ_g around it — and then re-score. Concretely:

1. Pick $\mathbf{x}^{(1)} = \arg\max_{\mathbf{x}} a(\mathbf{x})$.
2. Condition the GP variance on $\mathbf{x}^{(1)}$ (an exact rank-1 Cholesky update), lowering σ_g — and hence a — in its neighborhood.
3. Re-score the remaining pool and repeat until n points are chosen.

Because only the variance changes, the update is exact and cheap ($O(n \cdot m)$ per pick for m candidates), and the batch automatically spreads out to cover the weak *region* rather than a single weak *point*.

5.6 Fallbacks: when there is no trustworthy winner yet

Early on — before Phase 2 has produced a reliable winner, or when a residual fit fails numerically — the acquisition degrades gracefully down a ladder, and records which rung it used:

$$\texttt{residual\_ucb} \;\longrightarrow\; \texttt{gp\_variance} \;\longrightarrow\; \texttt{maximin\_fallback}. \tag{25}$$

gp_variance (the ALM criterion). Without a winner to critique, we fall back to a model-agnostic notion of "where is the field itself most uncertain". Fit one GP per PCA component (an independent emulator) and score a candidate by the total predictive variance of the *reconstructed* field. Because the PCA basis is orthonormal, that pixel-summed variance collapses **exactly** to a weighted sum of per-component variances — no sampling, no approximation:

$$\sum_{\text{pixels}} \mathrm{Var}\left[\hat{\psi}(\mathbf{x})\right] \;=\; \sum_{j=1}^{k} s_j^2\, \sigma_j^2(\mathbf{x}), \tag{26}$$

where s_j is the train-set standard deviation of coefficient j and $\sigma_j^2(\mathbf{x})$ is that component's posterior variance. This is the classical Active-Learning-MacKay (ALM) criterion, selected greedily with the same exact variance conditioning and the same feasibility weighting.

maximin_fallback. With too few successful solves to trust *any* GP (< 8), we give up on error modeling entirely and just spread out: greedy feasibility-weighted **maximin space-filling** in standardized input space, where each new point maximizes its (feasibility-weighted) distance to everything attempted so far:

$$\mathbf{x}^{(t+1)} = \underset{\mathbf{x}\in\text{pool}}{\arg\max}\; P_{\text{feas}}(\mathbf{x}) \cdot \min_{\mathbf{x}'\in\text{chosen}\,\cup\,\text{attempted}} \|\mathbf{x} - \mathbf{x}'\|. \tag{27}$$

Even this floor is better than another blind LHS batch, because it still avoids known-infeasible regions and never re-samples where we already have data.

5.7 Honest evaluation: the frozen envelope and per-cell errors

Active learning creates a measurement problem. Once we sample geometries *outside* the seed box, a test set carved from the seed dataset no longer represents the geometries we care about — it measures accuracy on the wrong distribution. The fix is a single **frozen, full-envelope evaluation set**: n_{eval} points (default 256) are drawn once by LHS over the *target envelope* (the seed box widened and clipped to physically sane values), solved, and never grown, so every winner across every meta-iteration is judged on identical samples. The acquisition draws over that same envelope, so the distribution we *evaluate* on and the distribution we *acquire* in agree.

There is also a subtler point: an aggregate RMSE can hide exactly the failure active learning is meant to fix. "Great in the seed box, broken at high κ" and "uniformly mediocre" can share the same average. So scoring is **stratified per geometry cell**: each eval sample is binned into a joint cell (n_b bins per dimension $\Rightarrow n_b^5$ cells), plus per-parameter tercile marginals, and RMSE is computed per occupied cell. The headline quantities are

$$\text{worst\_cell\_rmse} = \max_{c\in\text{occupied}} \text{RMSE}_c, \qquad \text{mean\_cell\_rmse} = \frac{1}{|\mathcal{C}|}\sum_{c\in\mathcal{C}} \text{RMSE}_c, \tag{28}$$

the latter weighting every geometry region equally regardless of how many samples landed in it. An early-stopping bar can even be phrased per cell — stop only once *every* occupied cell beats its own local baseline by the target margin, $\min_c 100\,(1 - \text{RMSE}_c \,/\, \text{RMSE}_c^{\text{base}}) \geq \tau$ — which directly rewards fixing the weakest region.

5.8 Proving it works: the control arm

Active sampling should be *demonstrably* better than blind sampling, not just plausibly so. The blind-LHS action `regen_dataset` is therefore retained unchanged as an **A/B control**. The headline experiment fixes the seed dataset and the budgets and compares `enrich_active` against `regen_dataset` on the same frozen envelope eval set, judged on **worst-cell RMSE reduction per OFT solve spent**.

6 The meta-loop — outer optimization by an LLM agent

6.1 The decision problem

Everything so far — generate data, fit a model, actively acquire — are *actions*. The meta-loop is what decides, at each step, which action to take. It is a sequential decision process (Figure 6): at iteration t the system computes a deterministic **diagnosis** of the current state (dataset statistics, winner metrics, per-cell errors, feasibility rates), the LLM reads that diagnosis and emits one **typed action** u_t from a small vocabulary, the action is dispatched by deterministic code, and the resulting state is scored — then the cycle repeats. The action space is

$$u_t \in \mathcal{A} = \{\ \texttt{regen\_dataset},\ \texttt{enrich\_active},\ \texttt{extend\_search},\ \texttt{terminate}\ \}: \tag{29}$$

`regen_dataset(overrides)` reshapes or resamples the dataset (blind LHS — the control arm); `enrich_active(`$n_{\texttt{new}}$`, strategy,` β`)` runs the active-learning acquisition of Section 6; `extend_search(focus)` runs a fresh Phase-2 AutoML round with directives (which models to emphasize, which hyperparameters to widen, the trial budget); and `terminate(reason)` stops. Crucially the action is emitted as *validated structured JSON*, so the action space is small, typed, and checked before anything runs — the LLM cannot invent an unsupported action or a malformed payload. The loop halts when the agent terminates, when an early-stopping quality bar is met (absolute RMSE, RMSE ratio, aggregate accuracy, or the strict per-cell worst-cell accuracy), or when a max-iteration safety cap is reached.

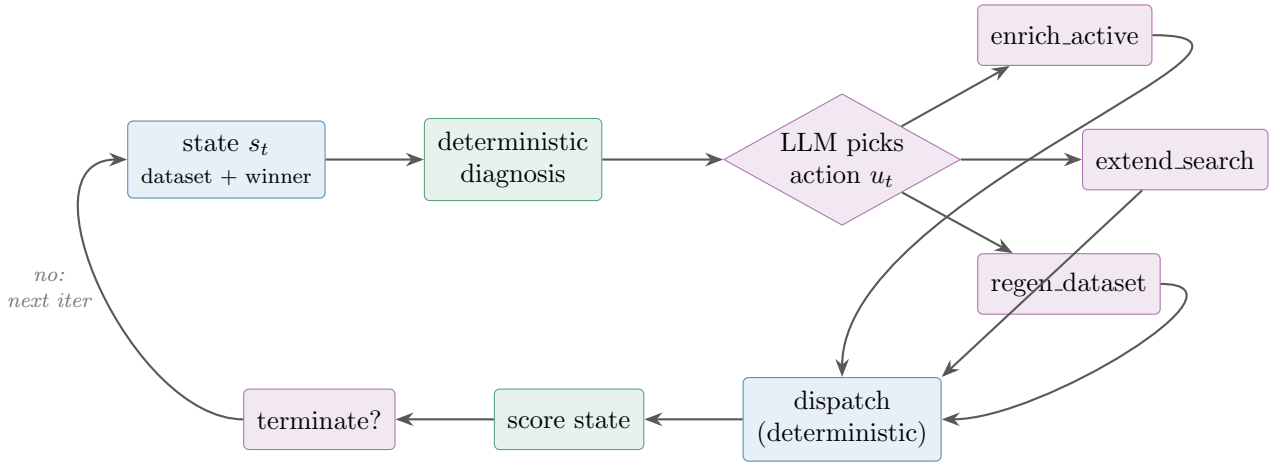

Figure 6: The meta-loop decision cycle. A deterministic diagnosis summarizes the state; the LLM (the only learned component) emits a typed action; deterministic code dispatches it and re-scores; the loop continues until the agent terminates or a quality bar / iteration cap is hit.

6.2 Division of labor: what is learned vs. what is deterministic

The design draws a hard line between judgment and computation, so that the science stays reproducible while the LLM still does something genuinely open-ended:

Job	Owner
Which meta-action to take, and its knobs (n_{new}, β)	LLM (DSPy module, GEPA-optimizable)
Which nested Phase-2 round to run	LLM (DSPy module)
Acquisition core (residual GP, UCB, batch, feasibility)	Deterministic library
Diagnosis of current state	Deterministic library
Open-ended representation / data-engineering search	URSA code-generation agent

Everything numerically load-bearing — the GP kernels, the UCB arithmetic, the CV folds — is deterministic library code; only the *high-level choices* are the LLM surface. This is what lets the platform claim reproducible results while still being "agentic".

6.3 Grading the agent: the meta-objective and GEPA

How do we know a whole meta-run was *good*, and how do we improve the agent that produced it? Each run is graded by a composite objective that first checks correctness (**hard gates**, all of which must pass for any credit) and then rewards quality:

$$S \;=\; \Big(\prod_{g\in\text{gates}} \mathbb{1}[g \text{ passes}]\Big)\cdot \sum_q \; w_q\, Q_q, \qquad \sum_q w_q = 1. \tag{30}$$

The gates verify that the deliverables exist, the report parses, an iteration log is present, and the winner actually loads and predicts — a run that fails any of these scores zero, no matter how good its numbers look. The quality terms then encode what "good judgment" means:

Term	Meaning	w_q
`final_rmse_vs_baseline`	$1 - \text{RMSE}_{\text{final}} / \text{RMSE}_{\text{base}}$, clipped to $[0,1]$	0.35
`improvement_over_iterations`	$(\text{RMSE}_{\text{first}} - \text{RMSE}_{\text{last}}) / \text{RMSE}_{\text{first}}$	0.20
`budget_efficiency`	fraction of total improvement by the halfway iteration	0.10
`no_waste`	fraction of post-first-winner iterations improving RMSE $\geq 1\%$	0.15
`terminated_by_agent`	decisive stop (agent-chosen or target-reached) vs. cap-riding	0.15
`runner_cleanliness`	only valid action types were used	0.05

Note what these weights buy: a decisive short run that stops once it is good scores *higher* than a budget-burning run that reaches the same accuracy but keeps churning — an explicit pressure toward efficiency, not just accuracy. Finally, the agent itself is improved against this score. Its decision module is written in **DSPy**, and its prompt/instructions are optimized by **GEPA**, a reflective prompt-evolution optimizer that searches instruction variants to maximize the expected score $\mathbb{E}[S]$ across recorded meta-traces. DSPy is retained *only* for this GEPA-optimizability (and as a clean way to write typed LLM programs); the numerical pipeline does not depend on it.

7 The agent layer (URSA) in more detail

The two layers of Figure 1 deserve a closer look at the orchestration side. **Layer 1 (`core`)** is the stable physics/IO substrate: config schema, LCFS + mesh construction, the TokaMaker solve

with isoflux-fallback retry, diagnostic scalar extraction (R_{axis}, q_0, q_{95}, ...), atomic file writes, and ordered logging. It carries one hard invariant that shapes all parallelism: `OFT_env` is a **process-wide singleton** (only one per Python interpreter), so any fan-out over solves must be process-level (subprocess or `spawn`), never threads — a constraint the orchestration layer respects everywhere.

Layer 2 (`agent`) is the URSA plan/execute machinery, the prompt specifications it consumes, the meta-loop orchestrator, and the DSPy scorers/modules. It offers two runner styles. In *structured* mode the deterministic library runs the pipeline and the LLM makes one typed decision per round — maximally reproducible. In *codegen* mode a URSA agent actually *writes* the runner script: a `PlanningAgent` emits a sequence of steps, an `ExecutionAgent` writes code for each, runs it, inspects the output, and threads a summary into the next step, and a feedback variant re-invokes the planner with the execution history so it can patch failures. The agent's genuinely open-ended role is the **representation / data-engineering search**: under a fixed evaluation protocol it reads the model diagnostics and authors and evaluates alternative input featurizations and output transforms — the one part of the pipeline deliberately left to code-generating LLM search rather than to a fixed library. Every run also writes an `experiments/<run_id>/trace.json` and a self-contained HTML report for inspection.

8 Summary

Autotokamak treats an expensive FEM equilibrium solver as a black-box forward operator $\mathcal{F} : \mathbb{R}^5 \to \psi(R, Z)$ and learns a fast surrogate $\hat{\mathcal{F}}$ by (i) reducing the flux field to a handful of PCA coefficients, (ii) regressing each coefficient from the shaping parameters with a small zoo of models (GP, kernel-ridge, poly-ridge, MLP), (iii) tuning hyperparameters by TPE Bayesian optimization against cross-validated field RMSE, and (iv) actively growing the training set with a residual-driven UCB acquisition that fits a GP error model to the winner's out-of-fold residuals, discounts by a GP-classifier feasibility model, and selects a diverse batch by exact kriging-believer variance conditioning. An outer meta-loop — whose high-level decisions are made by a DSPy/GEPA-optimized LLM and whose numerics are deterministic library code — iterates these phases against a frozen full-envelope, per-geometry-cell evaluation set, measuring and closing exactly where the surrogate is weak.

The recurring mathematical objects — the Grad–Shafranov PDE, the orthonormal PCA basis, the GP posterior mean/variance, the cross-validated RMSE objective, the UCB acquisition, and the gated composite meta-score — are the load-bearing formulas of the entire system, and the diagrams above (architecture, three-phase flow, surrogate factorization, active-learning loop, UCB intuition, and the meta-loop cycle) are the map that ties them together.